SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr DUMPALA PRASANTH

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Scenario-Based Assessment **DURATION: 1.5 Hrs**

1. A mobile app company wants to improve user engagement by showing personalized notifications at the right time. The company plans to use Reinforcement Learning to learn user behavior patterns. Report how the RL framework can be applied by defining the possible states, actions, and reward functions for this scenario.
(5 Marks)

1. Understanding of the Scenario

The mobile app company's goal is to increase user engagement by delivering personalised notifications at the moment a given user is most likely to respond positively. This is not a one-shot prediction problem; it is a sequential decision-making problem because the same user receives many notifications over time, each decision affects the user's future receptiveness (notification fatigue), and the true outcome (engagement or annoyance) is observed only after the action is taken. These three properties -- sequential decisions, delayed feedback, and an environment that reacts to the agent's actions -- are the defining characteristics of a Markov Decision Process (MDP), which makes Reinforcement Learning (RL) a natural and suitable framework for this scenario. The RL agent in this case is the notification engine, the

environment is the user together with the app usage context, and the objective is to learn a policy that maximises long-term cumulative engagement rather than a single immediate click.

2. RL Framework Definition

State Space (S):

The state must capture everything relevant to deciding when/whether to notify a specific user at a specific moment. A representative state vector is:

s = { time_of_day, day_of_week, minutes_since_last_app_open, recent_session_frequency (last 7 days), device_status (active/idle, screen on/off), location_context (home/work/travel), notification_history (count sent in last 24h, last response type), user_behaviour_cluster (derived from clustering past usage), current_app_feature_usage }

Action Space (A):

A = { send_now, delay_by_T_minutes, do_not_send, send_with_content_variant_k }. In the simplest binary formulation A = {send, do_not_send}; in a richer contextual-bandit/RL formulation the action also selects which notification content/category to use.

Reward Function (R):

R(s,a) = w1 . I(open within Δt) + w2 . I(in-app action after open) − w3 . I(ignored) − w4 . I(app muted notifications) − w5 . I(app uninstalled)

Typical weight assignment: w1 = +1 (click/open), w2 = +2 (deeper engagement such as purchase or content view), w3 = −0.1 (ignored, a small penalty to discourage spamming), w4 = −5 (user disables notifications), w5 = −20 (uninstall, the worst outcome). This shaped reward prevents the agent from optimising purely for short-term clicks at the expense of long-term retention.

Transition Dynamics:

P(s'|s,a) is not known in closed form because user behaviour is stochastic and non-stationary (it changes with season, app updates, personal habits); the agent must therefore learn from experience (model-free RL) rather than by planning with a known transition model.

3. Application of Concept -- Policy-Based Learning

Since CO5 emphasises policy-based reinforcement learning, the notification-timing decision is modelled using a stochastic policy $\pi\theta(a|s)$, parameterised by a neural network with weights θ, which outputs the probability of taking each action given the state. The policy is optimised directly (rather than deriving actions indirectly from Q-values) using the policy-gradient theorem:

$$\nabla\theta\, J(\theta) = E_{\tau\sim\pi\theta}\, [\, \Sigma_t\, \nabla\theta \log \pi\theta(a_t|s_t) \cdot G_t \,]$$

where $J(\theta)$ is the expected cumulative reward, τ is a trajectory of (state, action, reward) generated by following the policy, and $G_t = \Sigma_{k=0}^{\infty} \gamma^k r_{t+k}$ is the discounted return from time t ($\gamma \in [0,1)$ is the discount factor that down-weights distant future engagement relative to immediate engagement). The REINFORCE update rule applied at each training step is:

$$\theta \leftarrow \theta + \alpha \cdot \nabla\theta \log \pi\theta(a_t|s_t) \cdot G_t$$

where α is the learning rate. To reduce the high variance typical of REINFORCE, a baseline b(s_t) (often a learned value function V(s_t)) is subtracted from G_t, giving the advantage A_t = G_t − V(s_t), and the resulting Actor-Critic update becomes $\theta \leftarrow \theta + \alpha \cdot \nabla\theta \log \pi\theta(a_t|s_t) \cdot A_t$. This Actor-Critic formulation is well suited to the notification problem because it can update the policy continuously as new engagement data streams in, rather than waiting for full episodes to complete.

Simplified Training Loop (Pseudocode):

Initialise policy network $\pi\theta$ and value network $V\varphi$

For each incoming user event (state s):

Sample action $a \sim \pi\theta(a|s)$ [send / delay / suppress]

Observe reward r (click, ignore, uninstall signal) and next state s'

Compute advantage $A = r + \gamma \cdot V\varphi(s') - V\varphi(s)$

Update critic: $\varphi \leftarrow \varphi + \beta \cdot \nabla\varphi (r + \gamma \cdot V\varphi(s') - V\varphi(s))^2$

Update actor: $\theta \leftarrow \theta + \alpha \cdot \nabla\theta \log \pi\theta(a|s) \cdot A$

End for

4. Key Issues Identified

• Reward sparsity and delay: engagement signals are infrequent relative to the number of decisions made, slowing learning.

• Non-stationarity: user preferences and app-usage habits drift over time (e.g., holidays, new app features), so a policy trained on old data can become stale.

• Notification fatigue and cold-start: new users have no history, and over-notifying any user degrades long-term retention even if it briefly raises short-term click-through rate.

• Credit assignment: it can be difficult to know whether a later app-open was caused by the notification or by an unrelated event.

5. Decision / Recommendation

It is recommended to deploy the policy-gradient/Actor-Critic agent with (a) periodic offline retraining on a rolling data window to counter non-stationarity, (b) off-policy evaluation using importance sampling before any live rollout to estimate expected impact safely, (c) a hard cap on notification frequency per user per day to bound the fatigue risk regardless of what the learned policy suggests, and (d) A/B testing the RL policy against the current rule-based baseline before full production rollout.

6. Conclusion

By defining a context-rich state space, a compact action space, and a carefully shaped reward function that balances short-term engagement against long-term retention, this scenario can be effectively modelled and solved as a policy-based RL problem, delivering personalised notification timing that improves engagement while safeguarding against user fatigue and churn.

2. A warehouse robot uses Reinforcement Learning to find the shortest path for delivering packages while avoiding obstacles. Sometimes the robot chooses longer paths due to insufficient exploration during training. Solve how the exploration vs exploitation trade-off affects the robot's navigation performance and suggest some improvements.
(5 Marks)

1. Understanding of the Scenario

The warehouse robot must repeatedly navigate from a pick-up point to a drop-off point while avoiding obstacles. During training, the robot builds up value estimates (e.g., Q-values) for state-action pairs representing possible moves. The exploration-exploitation trade-off governs how the robot balances (a) exploiting its current best-known path (based on the Q-values learned so far) versus (b) exploring alternative, possibly better but currently unverified paths.

2. Effect on Navigation Performance

Exploitation means the robot always takes the action a* = argmax_a Q(s,a) that currently looks best. If the robot exploits too early and too often (i.e., explores too little), it locks onto the first path that gave acceptable results and never tests genuinely shorter alternatives, so its Q-values for the untested (better) path remain at their initial (often pessimistic or zero) values and are never corrected -- this is the direct cause of the robot repeatedly choosing a longer path, described in the scenario as 'insufficient exploration'. Conversely, if the robot explores too much (near-random movement for too long), it wastes time and battery repeatedly re-testing paths that are already known to be poor, delaying convergence to an efficient delivery policy. The correct balance allows the robot to (i) discover the truly shortest obstacle-free path and (ii) converge to consistently using it once found -- both are required for good navigation performance (short delivery time, low energy use, and few obstacle collisions).

3. Formal RL View

ε-greedy Action Selection:

$a = \{$ random action from A, with probability ε ; argmax_a Q(s,a), with probability $1 - \varepsilon \}$

Q-learning Update Rule (off-policy, model-free, tabular case suitable for a discretised warehouse grid):

$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \,]$

where $\alpha \in (0,1]$ is the learning rate controlling how much new information overrides old estimates, $\gamma \in [0,1)$ is the discount factor weighting future rewards, r is the immediate reward (e.g., −1 per step to encourage shortest paths, −100 for hitting an obstacle, +100 for reaching the destination), s' is the next state, and max_a' Q(s',a') is the best estimated value achievable from the next state.

Decaying Exploration Rate:

$\varepsilon_t = \varepsilon_{min} + (\varepsilon_{max} - \varepsilon_{min}) \cdot e^{(-\lambda \cdot t)}$

where t is the training step/episode number and λ controls how quickly exploration decays from ε_max toward ε_min. This ensures the robot explores heavily when it knows little about the warehouse layout, and gradually shifts toward exploitation as its Q-value estimates become reliable.

4. Suggested Improvements (with Justification)

Decaying ε-greedy exploration:

Start with a high ε (e.g., 0.9) so the robot broadly samples the warehouse layout early, then decay it toward a small floor value (e.g., 0.05) so it still occasionally re-checks for changes (e.g., new obstacles) without wasting excessive time later in training.

Softmax / Boltzmann exploration:

Instead of choosing uniformly at random when exploring, weight action choice by $P(a|s) = \exp(Q(s,a)/\tau) / \Sigma_b \exp(Q(s,b)/\tau)$, where τ is a temperature parameter. This biases exploration toward actions that already look moderately promising rather than fully random moves, speeding up discovery of good paths.

Upper Confidence Bound (UCB) action selection:

$a = \text{argmax}_a [Q(s,a) + c \cdot \text{sqrt}(\ln(t) / N(s,a))]$, where N(s,a) counts how often action a has been tried in state s. This automatically directs exploration toward under-tried actions rather than purely random ones, improving sample efficiency.

Optimistic Initialisation:

Initialise all Q(s,a) to a high value so that every untried path initially looks attractive; the robot is then naturally drawn to explore paths it has not yet tried, which corrects the estimates quickly.

Intrinsic Curiosity / Exploration Bonus:

Add a small bonus reward for visiting states the robot has rarely seen, $r_{total} = r_{extrinsic} + \beta \cdot r_{intrinsic}(s)$, encouraging systematic coverage of the warehouse map, particularly useful when the layout is large.

Entropy Regularisation (if a stochastic policy-gradient method is used instead of tabular Q-learning):

Add an entropy bonus to the objective, $J(\theta) = E[\Sigma r_t] + \beta \cdot H(\pi\theta(\cdot|s))$, which discourages the policy from collapsing to a single deterministic path too early, preserving useful exploration.

5. Decision / Recommendation

A combination of decaying ε-greedy (or UCB for more principled exploration) with optimistic initialisation is recommended for a discretised warehouse grid, since it is simple to implement, computationally cheap for real-time robot control, and empirically reduces the number of training episodes needed to discover and settle on the shortest safe path.

6. Conclusion

The exploration-exploitation trade-off directly determines whether a warehouse robot's learned navigation policy converges to a globally efficient path or gets stuck exploiting a suboptimal one. Structured exploration strategies (decaying ε-greedy, UCB, optimistic initialisation, or

entropy regularisation) resolve the issue described in the scenario by ensuring the robot sufficiently samples alternative routes before committing to a final policy.

3. A hospital is considering using Reinforcement Learning to optimize patient appointment scheduling in order to reduce waiting time and improve resource utilization. Demonstrate the suitability of RL for this scenario and discuss possible challenges such as delayed rewards and ethical considerations.
(5 Marks)

1. Understanding of the Scenario

Hospital appointment scheduling involves assigning patients to available time slots and resources (doctors, rooms, equipment) under uncertainty -- patients may arrive late, miss appointments (no-shows), or require emergency prioritisation, and resource availability can change dynamically. The objective is to minimise patient waiting time while maximising resource utilisation. Because decisions must be made repeatedly and their consequences unfold over time and depend on prior decisions, this is fundamentally a sequential decision-making problem under uncertainty, which is exactly the class of problem Reinforcement Learning is designed to solve.

2. Suitability of RL for This Scenario

Unlike static optimisation techniques that solve one fixed snapshot of the schedule, an RL agent continuously interacts with the live hospital environment, observes outcomes of its scheduling decisions (waiting times, no-shows, resource idling), and updates its policy accordingly. This allows the system to adapt automatically to seasonal patient-flow changes, evolving no-show patterns, and newly added resources -- something a one-off optimisation model cannot do without being manually re-solved. RL is therefore highly suitable, provided ethical and safety constraints are respected.

3. RL Formulation

State Space (S):

s = { number of patients currently waiting, doctor/room/equipment availability, each waiting patient's urgency/priority score, historical no-show probability for the time slot, time of day, current queue length per department }

Action Space (A):

A = { assign patient to slot X with resource Y, place patient on a waitlist, reschedule to a later slot, immediately prioritise an urgent/emergency case }

Reward Function (R):

$R = -\alpha \cdot$ (average patient waiting time) $- \beta \cdot$ (resource idle time) $+ \gamma \cdot$ (utilisation efficiency) $- \delta \cdot$ (missed/critical-case delay penalty)

A concrete example: $R = -0.5 \cdot (\text{waiting\_minutes}) - 0.3 \cdot (\text{idle\_minutes}) + 0.2 \cdot (\text{slots\_filled/total\_slots}) - 5 \cdot (\text{critical\_case\_delay\_flag})$. The large penalty term for delaying a critical case ensures the reward structure never incentivises the agent to trade patient safety for efficiency.

Suitable Algorithm:

Given the discrete, resource-constrained nature of scheduling with clear operational constraints, a Constrained Markov Decision Process (CMDP) solved via a policy-based method (e.g., Constrained Policy Optimisation, or Lagrangian Actor-Critic) is appropriate: the agent maximises expected reward subject to hard constraints (e.g., 'maximum waiting time for an urgent case $\leq$ 15 minutes') enforced via a Lagrangian penalty term added to the objective: $L(\theta,\lambda) = J(\theta) - \lambda \cdot$ (constraint violation).

4. Key Challenges

Delayed Rewards:

The full benefit of a scheduling decision -- for example, whether assigning a patient to a particular time actually improved health outcomes or reduced total system congestion -- is frequently observed only much later (sometimes after the appointment itself, or after treatment outcomes are known), creating a classic credit-assignment problem: it becomes difficult to know precisely which earlier scheduling decision is responsible for a later good or bad outcome. This delay can slow convergence and make the learning signal noisy, especially in a system as variable as a hospital.

Ethical Considerations:

- Fairness: the policy must not systematically disadvantage particular patient groups (by age, insurance status, ethnicity, or gender) when assigning slots.
- Priority handling: genuine medical emergencies must always override efficiency-driven scheduling, which requires the constraint to be treated as a hard rule, not merely a heavily-weighted soft reward term.
- Transparency and accountability: clinicians and patients should be able to understand why a particular scheduling decision was made, especially if a complaint arises.
- Data privacy: patient health and scheduling data used to train the RL agent must be protected under data-protection regulations (e.g., HIPAA/GDPR-equivalent standards).
- Over-automation risk: fully removing human oversight from a healthcare-critical process is risky; a human-in-the-loop safety net is necessary.

5. Decision / Recommendation

Deploy the RL scheduler as a decision-support system with a human supervisor able to override any assignment, formalise emergency priority as a hard constraint using a Constrained MDP rather than a soft reward penalty, validate the policy extensively in a simulated hospital environment before live deployment, and conduct periodic fairness audits comparing waiting-time distributions across demographic groups.

6. Conclusion

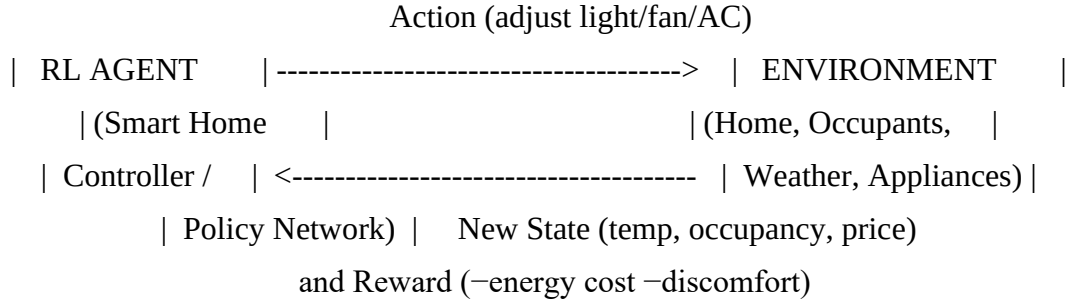
Reinforcement Learning is well suited to hospital appointment scheduling because of its ability to adapt to a dynamic, uncertain environment, but its use must be carefully engineered around delayed-reward challenges and strong ethical safeguards, particularly regarding fairness, transparency, and the non-negotiable priority of medical emergencies.

4. A smart home system aims to reduce electricity consumption by automatically controlling lights, fans, and air conditioners based on user presence and weather conditions. Sketch a Reinforcement Learning model for this scenario by defining state space, action space, reward structure, and suitable learning algorithm.
(5 Marks)

1. Understanding of the Scenario

The smart home system must continuously decide how to control lights, fans, and air conditioners to minimise electricity consumption while maintaining occupant comfort, based on presence and weather signals that change throughout the day. Because the system must make a stream of interdependent control decisions whose consequences (energy cost, comfort) unfold over time, this is naturally modelled as a Reinforcement Learning problem in which the smart home controller is the agent, and the physical house plus its occupants form the environment.

2. Agent-Environment Interaction Diagram (Sketch)



This closed-loop diagram represents the standard MDP interaction cycle: the agent observes a state, takes an action, and receives a new state and a reward, repeating this cycle continuously.

3. RL Model Definition

State Space (S):

s = { occupancy/presence sensor status, indoor temperature, indoor humidity, outdoor temperature forecast, current appliance states (light level, fan speed, AC setpoint), time of day, electricity tariff rate at that time }

Action Space (A):

A = { light dim level ∈ [0,100]%, fan speed ∈ {off, low, medium, high}, AC target temperature ∈ [18°C, 28°C] }. Because several of these are continuous or multi-level, this is a continuous/hybrid action space rather than a small discrete one.

Reward Structure (R):

$R = -\lambda 1 \cdot (\text{energy\_cost\_t}) - \lambda 2 \cdot (\text{discomfort\_penalty\_t}) + \lambda 3 \cdot (\text{comfort\_bonus\_t})$

where $\text{discomfort\_penalty\_t} = |T_{actual} - T_{preferred}|$ (temperature deviation from occupant preference) and $\text{energy\_cost\_t} = \text{tariff\_rate\_t} \times \text{power\_consumed\_t}$. The weights $\lambda 1$, $\lambda 2$, $\lambda 3$ let the system designer tune the trade-off between saving energy and maintaining comfort.

4. Suitable Learning Algorithm and Justification

Because the action space includes continuous variables (dimming level, AC setpoint), tabular Q-learning (which requires discrete, enumerable state-action pairs) becomes impractical due to the curse of dimensionality. A policy-based / actor-critic deep RL method is far more suitable:

Proximal Policy Optimisation (PPO):

PPO optimises a clipped surrogate objective $L(\theta) = E_t[\ \min(\ r_t(\theta)\cdot A_t,\ \text{clip}(r_t(\theta),\ 1-\varepsilon,\ 1+\varepsilon)\cdot A_t\ )\ ]$, where $r_t(\theta) = \pi\theta(a_t|s_t) / \pi\theta\_old(a_t|s_t)$ is the probability ratio between the new and old policy, and A_t is the advantage estimate. The clipping term prevents overly large, destabilising policy updates, which is valuable in a home environment where erratic control changes (e.g., AC flipping rapidly) would be undesirable and costly.

Deep Deterministic Policy Gradient (DDPG) / Actor-Critic:

DDPG maintains an actor network $\mu\theta(s)$ that outputs continuous control actions directly, and a critic network $Q\varphi(s,a)$ that evaluates them, updated via: Critic loss $= E[(r + \gamma\cdot Q\varphi'(s', \mu\theta'(s')) - Q\varphi(s,a))^2]$, Actor update: $\nabla\theta\ J \approx E[\nabla a\ Q\varphi(s,a)|_{a=\mu\theta(s)} \cdot \nabla\theta\ \mu\theta(s)]$. This directly outputs a real-valued setpoint (e.g., 'set AC to 23.4°C') rather than requiring discretisation.

Training Loop (Simplified Pseudocode):

Initialise actor $\pi\theta$, critic $Q\varphi$, replay buffer D

For each time step:

Observe state s (occupancy, temperature, tariff)

Select action $a = \pi\theta(s)$ + exploration noise

Apply a (adjust light/fan/AC); observe reward r and next state s'

Store (s,a,r,s') in replay buffer D

Sample mini-batch from D; update critic $Q\varphi$ and actor $\pi\theta$ using gradients above

End for

5. Key Issues and Practical Considerations

• Comfort vs cost trade-off tuning: incorrect reward weighting can either waste energy or make the house uncomfortable.

• Sensor noise and occupancy detection errors can mislead the agent into acting on incorrect presence information.

• Safety constraints: the AC/heater should never be allowed to reach unsafe temperature extremes regardless of what the policy outputs -- this requires an action-clipping/safety layer around the learned policy.

6. Decision / Recommendation

A PPO-based continuous-control policy is recommended for this scenario due to its training stability and native support for continuous action spaces, combined with a hard safety layer that clips any unsafe temperature or fan-speed command before it reaches the physical appliance.

7. Conclusion

Modelling the smart home as an MDP with a rich continuous state and action space, a comfort-aware reward function, and a policy-based algorithm such as PPO or DDPG enables the system to learn an energy-efficient yet comfortable control strategy that adapts to occupancy and weather in real time.

5. An online gaming platform uses Reinforcement Learning to match players of similar skill levels to maintain engagement. Report how reward design can influence matchmaking quality with reward definition that might lead to biased or inefficient matching results.
(5 Marks)

1. Understanding of the Scenario

An online gaming platform's matchmaking system must repeatedly decide how to pair players so that resulting matches are competitive and enjoyable, keeping players engaged with the platform over time. This is an RL problem because the matching policy affects not just the immediate match quality but also the player's future willingness to keep playing (long-term retention), so the system must be learned and optimised over many sequential matchmaking decisions.

2. RL Formulation

State Space (S):

s = { skill ratings of players in the queue (e.g., Elo/TrueSkill), queue waiting time so far, recent win/loss streaks, historical engagement level of each player, time of day/queue population size }

Action Space (A):

A = { pair player i with player j (from the candidate pool), widen or narrow the acceptable skill-difference window, hold/wait for a better match }

Reward Function (R):

$R = -|SkillA - SkillB| + \lambda 1 \cdot$ (post-match session continuation) $- \lambda 2 \cdot$ (player drop-out/complaint) $- \lambda 3 \cdot$ (excessive queue wait time)

3. How Reward Design Influences Matchmaking Quality

A well-designed multi-term reward, as above, encourages the agent to find a match that is simultaneously balanced (small skill gap), fast to arrange (bounded wait), and conducive to continued play. If any one term dominates the reward disproportionately, matchmaking quality degrades in predictable ways.

Case A -- Reward Skewed Toward Speed:

If R is defined mainly to minimise queue time (R = −wait_time only), the agent learns to pair players regardless of skill gap just to reduce queue length, producing lopsided, unfun matches (e.g., a beginner facing an expert), which directly harms player experience and retention despite technically 'succeeding' on the stated reward.

Case B -- Reward Skewed Toward Pure Skill-Matching:

If R is defined purely as R = −|SkillA − SkillB| with no penalty for wait time, the agent may hold players in queue indefinitely searching for a near-perfect skill match, producing excessively long wait times, which itself causes player drop-out -- an unintended and inefficient outcome.

Case C -- Reward Skewed Toward Raw Engagement/Session Length:

If the reward simply maximises how long a player keeps playing afterward, the agent could learn to exploit psychological hooks -- for example, repeatedly pairing a weaker player against a slightly stronger one to create a 'just barely losing' experience shown in some literature to be highly retention-inducing but ethically manipulative and damaging to the weaker player's enjoyment. This is a clear case of reward hacking, where the optimised metric diverges from genuine player welfare.

Case D -- Reward Ignoring Diversity:

A reward that never penalises repeated pairing of the same players narrows the effective matchmaking pool and can create in-group biases or perceived unfairness (e.g., always facing the same rival).

4. Formal Multi-Objective Reward Recommendation

R_total = w1·(−|SkillA − SkillB|) + w2·(−wait_time) + w3·(long_term_retention_indicator) − w4·(reward_hacking_flag, e.g., suspiciously repeated pairings)

with weights (w1..w4) tuned and periodically re-validated against real player-satisfaction surveys, not only proxy engagement metrics, to catch cases where the proxy diverges from genuine enjoyment.

5. Decision / Recommendation

Use the multi-objective reward above rather than any single-term reward, cap the maximum acceptable skill gap and maximum queue wait time as hard constraints (constrained optimisation) rather than only soft penalty terms, and run periodic audits of match outcomes (win-rate distribution, repeat-pairing frequency, player complaint rate) to detect and correct reward-hacking behaviour before it damages the player base.

6. Conclusion

Reward design is the single most influential factor in matchmaking quality: a narrow or poorly balanced reward function can produce biased, inefficient, or even manipulative matchmaking

outcomes, whereas a carefully weighted multi-objective reward -- combined with constraints and regular auditing -- produces balanced, fair, and genuinely engaging matches.

6. A financial trading system applies Reinforcement Learning to decide buy/sell actions based on market signals. Analyze how states, actions, and rewards can be defined in this context and evaluate the risks of delayed rewards in volatile markets.

(5 Marks)

1. Understanding of the Scenario

A financial trading system must repeatedly decide whether to buy, sell, or hold an asset based on incoming market signals, with the objective of maximising portfolio value over time. Because each trading decision affects future portfolio state (holdings, cash) and the market itself evolves stochastically, this is a sequential decision-making problem under uncertainty, well suited to an RL formulation, though it carries significant financial risk that must be carefully managed.

2. RL Formulation

State Space (S):

s = { recent price history/candlestick data, trading volume, technical indicators (moving averages, RSI, MACD, volatility index), current portfolio holdings, available cash balance, recent realised profit/loss }

Action Space (A):

A = { buy (with size), sell (with size), hold }. In an extended formulation the action can be a continuous value representing the fraction of capital to allocate.

Reward Function (R):

R_t = (Portfolio_Value_t − Portfolio_Value_(t-1)) − transaction_cost_t

A more robust, risk-aware alternative uses the Sharpe ratio as the reward basis:

Sharpe_Ratio = (Mean(Return) − Risk_Free_Rate) / StdDev(Return)

Using a risk-adjusted reward like the Sharpe ratio (rather than raw profit) discourages the agent from taking excessively volatile positions purely to chase high average returns.

Reward Computation (Pseudocode):

For each trading step t:

```
portfolio_value_t = cash_t + holdings_t * price_t
raw_return = portfolio_value_t − portfolio_value_(t-1)
reward = raw_return − transaction_cost
# risk-adjusted variant computed over a rolling window W of past returns:
reward_risk_adjusted = (mean(returns_W) − risk_free_rate) / (std(returns_W) + 1e-6)
```

3. Risks of Delayed Rewards in Volatile Markets

Credit Assignment Problem:

The true profitability of holding an asset is frequently realised only after a substantial time delay (e.g., a stock bought today may only show its real gain/loss weeks later); this makes it hard for the agent to correctly attribute later portfolio changes to the specific earlier buy/sell decision responsible for them.

Reward Noise Amplification:

In volatile markets, short-term price swings create large, noisy immediate rewards that do not reflect genuine long-term profitability, which can mislead a value-based agent into overfitting to transient market noise rather than learning a robust trading strategy.

Excessive Risk-Taking:

If the agent is rewarded on short time horizons, it may learn to take large, high-variance positions that occasionally produce big short-term rewards (reinforcing the risky behaviour) while exposing the portfolio to catastrophic drawdowns during adverse volatility spikes -- a clear example of the exploration-driven agent chasing spurious short-term reward.

Slow Convergence:

Because meaningful feedback is delayed, the agent needs many more interactions (trades) to converge to a stable, reliable policy, during which real capital may be at risk if trained live rather than in simulation.

4. Decision / Recommendation

Use a risk-adjusted, discounted reward (Sharpe-ratio based or Sortino-ratio based) rather than raw short-term profit; apply strict position-sizing limits and maximum drawdown constraints (treated as hard constraints in a constrained-RL formulation); train initially in a realistic market simulator/backtest environment before any live deployment; and prefer a policy-based Actor-Critic algorithm for continuous position-sizing decisions, since it can more smoothly incorporate risk penalties into the policy-gradient update than a purely value-based method.

5. Conclusion

Delayed and noisy rewards are an inherent risk in applying RL to volatile financial markets, and mitigating them requires deliberate reward shaping (risk-adjusted returns), hard risk constraints, and careful offline validation before any live capital is committed to the learned trading policy.

7. A drone delivery company uses RL to optimize delivery routes while avoiding restricted airspace. Construct a suitable RL framework by defining states, actions, and rewards, and justify how penalties can ensure safe navigation. (5 Marks)

1. Understanding of the Scenario

A drone delivery company needs its drones to autonomously plan and follow delivery routes that are efficient (fast, low battery use) while strictly avoiding restricted or legally prohibited

airspace. Because the drone continuously makes navigation decisions in a dynamic environment (wind, obstacles, changing no-fly zones) with clear safety-critical constraints, this is well suited to a constrained Reinforcement Learning formulation.

2. RL Framework for Drone Delivery

State Space (S):

s = { drone GPS position (x,y), altitude, current battery level, wind speed/direction, distance and bearing to delivery destination, proximity to nearest restricted-airspace boundary, presence of dynamic obstacles }

Action Space (A):

A = { move north/south/east/west, ascend, descend, hover, adjust speed }, typically discretised into a fixed set of directional/altitude commands issued at each control cycle.

Reward Function (R):

R = R_delivery − P_restricted − P_time − P_battery − P_collision

Where: R_delivery = large positive reward (e.g., +100) on successful on-time delivery; P_restricted = very large penalty (e.g., −200) for entering restricted airspace; P_time = small negative reward per timestep (e.g., −1) to encourage the shortest safe path; P_battery = penalty proportional to unnecessary energy expenditure; P_collision = large penalty for near-collision with an obstacle.

3. Constructing the Framework -- Constrained MDP View

Rather than relying purely on the soft reward penalty above, the restricted-airspace rule should additionally be encoded as a hard constraint in a Constrained MDP: maximise $E[\Sigma\ \gamma^t\ r_t]$ subject to $E[\Sigma\ \gamma^t\ c_t] \leq d$, where $c_t = 1$ if the drone is inside restricted airspace at time t (else 0), and d is set to (near) zero, meaning the expected fraction of time spent violating restricted airspace must not exceed a strict threshold. This is solved using a Lagrangian relaxation: $L(\theta,\lambda) = J(\theta) - \lambda \cdot (E[\Sigma\ c_t] - d)$, where λ is a dynamically adjusted multiplier that increases the effective penalty whenever the agent's policy violates the constraint too often during training, forcing the learned policy to respect the boundary robustly rather than merely 'discouraging' it.

4. Justification -- Why Large Penalties Ensure Safe Navigation

The restricted-airspace penalty (e.g., −200) is deliberately set far larger than any possible efficiency gain (a shortcut could save at most a small number of time-penalty units, e.g., −1 per step for perhaps 10-20 steps). Because the expected return of any trajectory that enters restricted airspace is therefore always lower than a safe, slightly longer detour, the optimal policy under standard RL theory (which maximises expected cumulative reward) will never choose to violate the restriction once the value function has converged. Formalising this additionally as a hard constraint via the Lagrangian method above provides a stronger, more provable guarantee than reward shaping alone, since the constraint violation probability itself becomes part of the optimisation objective, not just an indirect side-effect of reward magnitude.

Safety Layer / Shielding (Additional Practical Safeguard):

In practice, deployed drone systems typically add a rule-based 'shield' outside the learned policy: any action that the learned policy proposes is checked against a geofence boundary

before execution, and is automatically overridden/blocked if it would cause a restricted-airspace violation, providing a deterministic safety guarantee independent of how well the RL policy has converged.

5. Key Issues

• GPS/localisation inaccuracy near airspace boundaries could cause unintended violations even with a correct policy.

• Dynamic no-fly zones (e.g., temporary restrictions for events) require the state representation to be updated in real time.

• Battery constraints may conflict with a longer, safer detour route, requiring careful reward-weight tuning.

6. Decision / Recommendation

Combine (a) a heavily-weighted reward penalty for restricted-airspace entry, (b) a formal Constrained MDP/Lagrangian enforcement of a near-zero violation probability, and (c) a deterministic rule-based geofencing shield as a final safety net, so that airspace safety does not depend solely on the learned policy having fully converged.

7. Conclusion

A constrained RL framework, in which airspace-restriction violations are penalised heavily and enforced as a formal constraint (not just a reward term), combined with a deterministic safety shield, provides drone delivery systems with efficient route optimisation while guaranteeing the strict avoidance of restricted airspace required for safe operation.

8. A university applies RL to personalize online learning paths for students. Examine how reward design can influence student engagement and defend the ethical considerations of adapting learning content dynamically. (5 Marks)

1. Understanding of the Scenario

The university wants to personalise online learning paths for students, dynamically choosing which content, difficulty level, or resource to present next based on each student's ongoing performance and engagement. This is a sequential decision-making problem -- each content recommendation affects the student's subsequent learning state and engagement -- and therefore RL is a natural fit, provided ethical safeguards are respected given the sensitive, human-centred nature of education.

2. RL Formulation (Context)

State Space (S):

s = { current topic mastery level, recent quiz/assessment scores, time spent per topic, engagement indicators (session frequency, completion rate), learning style/preference signals }

Action Space (A):

A = { present easier content, present harder/advanced content, present a different content format (video/text/practice), recommend a review module, advance to next topic }

Reward Function (R):

R = w1 · (concept mastery improvement) + w2 · (appropriate time-on-task) + w3 · (continued session participation) − w4 · (student disengagement/drop-out signal)

3. Influence of Reward Design on Student Engagement

Well-Designed Reward:

When the reward jointly captures mastery improvement and healthy engagement (not just raw completion speed), the RL policy learns adaptive pacing: presenting slightly challenging but achievable content, which research in education consistently shows keeps students in a productive 'zone of proximal development', improving both learning outcomes and sustained engagement.

Poorly-Designed Reward -- Case A (Speed-Focused):

If R rewards only the number of modules completed or completion speed, the agent learns to recommend easier content to maximise apparent 'progress' metrics, which superficially looks successful but under-challenges students, reduces genuine learning, and eventually causes disengagement once students realise they are not being appropriately challenged.

Poorly-Designed Reward -- Case B (Raw Engagement/Time-on-Platform):

If R rewards only how long a student stays active on the platform, the agent may push excessive or repetitive content purely to inflate session length, which can cause frustration, burnout, and ultimately higher drop-out -- a case of the proxy metric diverging from the true educational goal (learning), i.e., reward hacking.

4. Ethical Considerations

Bias and Fairness:

If historical training data under-represents certain student groups (e.g., students from under-resourced schools), the learned policy may systematically recommend weaker or less challenging learning paths to those groups, reinforcing existing educational inequities.

Transparency and Explainability:

Students and instructors should be able to understand why a particular content path was recommended, both to build trust and to allow instructors to intervene if the recommendation seems inappropriate.

Data Privacy:

Detailed learning-behaviour data (time spent, mistakes made, engagement patterns) is sensitive and must be protected under data-protection regulations, with clear consent and minimal necessary retention.

Avoiding Manipulative Design:

The system must avoid exploiting psychological hooks purely to maximise engagement metrics (similar to addictive app design patterns), since the platform's purpose is genuine learning, not attention-capture.

Human Oversight:

A teacher or academic advisor should remain able to review and override the automated learning path, particularly for students showing signs of struggle or disengagement, since an algorithm alone should not have unchecked control over a student's educational trajectory.

5. Decision / Recommendation

Design the reward function to combine mastery-based learning outcomes with well-being-aware engagement indicators (not raw completion speed or time-on-platform alone), keep a human instructor in the loop for oversight and override, and conduct regular fairness audits comparing recommended learning-path difficulty/quality across demographic and socio-economic student subgroups to detect and correct systemic bias.

6. Conclusion

Reward design is central to whether an RL-based personalised learning system genuinely improves student engagement and outcomes or inadvertently narrows learning and introduces bias; careful, multi-objective reward design combined with strong ethical safeguards (fairness audits, transparency, privacy protection, human oversight) is essential for responsible deployment in an educational setting.

9. A traffic management authority deploys RL to control traffic lights at busy intersections. Illustrate how constraints such as emergency vehicle priority and pedestrian safety can be incorporated into the RL model and interpret the impact on congestion reduction.

(5 Marks)

1. Understanding of the Scenario

A traffic management authority wants to use RL to control traffic-light phases at busy intersections in order to reduce congestion, while still respecting critical safety and operational rules -- specifically, emergency-vehicle priority and pedestrian safety. Because traffic conditions vary continuously and signal decisions must be made repeatedly in response to real-time vehicle/pedestrian flow, this is a natural sequential decision-making (MDP) problem, but one that requires hard constraints layered on top of the standard reward-maximisation objective.

2. RL Model with Constraints

State Space (S):

s = { queue length and vehicle count per lane/approach, current signal phase and elapsed time in that phase, presence of an emergency-vehicle transponder signal, pending pedestrian crossing requests, time of day/traffic pattern }

Action Space (A):

A = { keep current phase, switch to next scheduled phase, extend current green duration, immediately trigger emergency-priority phase, trigger pedestrian-crossing phase }

Reward Function (R):

$R = -\alpha \cdot$ (total vehicle waiting time) $- \beta \cdot$ (queue length) $+ \gamma \cdot$ (bonus for fast emergency-vehicle clearance) $- \delta \cdot$ (penalty if pedestrian safety minimum is violated)

3. Incorporating Constraints into the RL Model

Emergency Vehicle Priority (Hard Constraint):

Rather than relying only on a large reward bonus, emergency-vehicle priority is best implemented as a deterministic pre-emption rule layered on top of the learned policy: whenever an emergency-vehicle signal is detected in the state, the system immediately overrides the RL-selected action and switches to the emergency-priority phase, regardless of what the learned policy would otherwise recommend. Formally, this can be expressed as a constrained MDP: maximise $E[\Sigma \gamma^t r_t]$ subject to $P(\text{emergency vehicle delayed beyond threshold } \tau) \approx 0$, enforced via either a hard pre-emption rule or a Lagrangian penalty with a very large multiplier $\lambda_{emergency}$.

Pedestrian Safety (Minimum Guarantee Constraint):

Pedestrian safety is incorporated as a minimum-walk-time constraint: once a pedestrian crossing phase begins, it must run for at least T_min seconds regardless of vehicle-side reward pressure to switch back early, and a mandatory all-red safety buffer is enforced between phase switches to allow the crossing to clear. These are treated as non-negotiable constraints in the action-selection logic, not as reward terms the agent might learn to trade off.

Simplified Control Loop with Constraints (Pseudocode):

For each control cycle:

```
    Observe state s (queue lengths, phase, emergency signal, pedestrian request)
    If emergency_vehicle_detected:
        action = EMERGENCY_PRIORITY_PHASE    # hard override, bypasses learned policy
    Else if pedestrian_request AND min_walk_time_not_yet_elapsed:
        action = MAINTAIN_PEDESTRIAN_PHASE   # hard constraint, cannot be shortened
    Else:
        action = πθ(s)   # normal reward-optimised RL policy decision
    Execute action; observe reward r and next state s'; update policy πθ
```

4. Impact on Congestion Reduction

Enforcing these constraints introduces a modest trade-off: average vehicle waiting time may increase slightly during periods with frequent emergency-vehicle events or pedestrian crossings, compared to a purely wait-time-optimised (unconstrained) policy. However, this trade-off is necessary and acceptable because it guarantees legal compliance and public safety, which cannot be sacrificed for marginal efficiency gains. Overall, despite these constraints, an adaptive constrained-RL traffic controller still meaingfully reduces average congestion relative to traditional fixed-time signal plans, because it continuously reallocates green time to the busiest approaches in real time, only yielding when a safety-critical event genuinely requires it.

5. Decision / Recommendation

Implement emergency-vehicle pre-emption and minimum pedestrian walk-time as hard, rule-based overrides sitting outside the learned policy (rather than only soft reward penalties), and let the RL policy optimise congestion only within the remaining decision space -- this combination maximises congestion reduction while providing a provable safety guarantee for the two critical constraints.

6. Conclusion

A constrained RL approach that treats emergency-vehicle priority and pedestrian safety as hard, non-negotiable rules -- while allowing the learned policy to freely optimise congestion within those bounds -- delivers a traffic-light control system that meaningfully reduces congestion without compromising the safety-critical requirements of the intersection.

10. A manufacturing plant uses RL to schedule machines under resource limitations. Differentiate between traditional optimization methods and constrained RL approaches, and assess which method provides better adaptability in dynamic production environments. (5 Marks)

1. Understanding of the Scenario

A manufacturing plant must schedule multiple machines to process jobs under limited resources (machine capacity, tooling, workforce, energy), while conditions such as machine breakdowns, urgent orders, and demand fluctuations can change dynamically. The task asks us to differentiate between classical/traditional optimisation methods and constrained Reinforcement Learning approaches for solving this scheduling problem, and to assess which is better suited to a dynamic production environment.

2. Traditional Optimisation Methods

Definition and Examples:

Traditional approaches include Linear Programming (LP), Mixed-Integer Programming (MIP), classical Job-Shop Scheduling heuristics, and metaheuristics such as Genetic Algorithms or Simulated Annealing. These methods formulate the scheduling problem explicitly: minimise total makespan or cost subject to constraints such as machine capacity limits, precedence relations between tasks, and due dates -- expressed mathematically, e.g., minimise Σ C_j (completion times) subject to resource-capacity and precedence constraints.

Characteristics:

They require an explicit, largely deterministic mathematical model of the problem (known processing times, fixed resource availability), solve a static snapshot of the schedule at a point in time, and typically must be re-solved from scratch whenever conditions change -- for example, a machine breakdown or an urgent new order arriving mid-shift requires re-running the entire optimisation, which can be computationally expensive for large-scale or highly

dynamic production floors. Their major strength is that, for well-defined, static problems, they can produce provably optimal or near-optimal solutions with strong theoretical guarantees.

3. Constrained Reinforcement Learning Approaches

Definition:

Constrained RL formulates the scheduling problem as a Constrained Markov Decision Process (CMDP): maximise $E[\Sigma\ \gamma^t\ r_t]$ subject to $E[\Sigma\ \gamma^t\ c_t] \leq d$, where c_t represents constraint violations such as exceeding machine capacity, and d is the allowed violation budget (often set to zero for hard operational limits). The RL agent (scheduler) learns, through repeated interaction with a simulated or real production environment, a policy that assigns jobs to machines/time-slots while respecting these constraints, typically enforced via Lagrangian relaxation, a safety-layer/shielding mechanism, or reward shaping with heavy constraint-violation penalties.

Characteristics:

Rather than solving one fixed instance, the agent learns a general policy that can react in real time to disruptions (machine breakdowns, new urgent orders, changing resource availability) without needing to be fully re-solved from scratch -- the trained policy simply observes the new state and outputs an updated scheduling decision. This adaptability comes at the cost of requiring substantial training data or a high-fidelity simulator, longer upfront training time, and generally lower interpretability/explainability compared to a mathematical programming solution.

4. Formal Comparison Table

- Dimension: Model Requirement -- Traditional Optimisation needs an explicit, largely deterministic mathematical model; Constrained RL learns from interaction/simulation, does not require a fully explicit closed-form model.
- Dimension: Adaptability to Change -- Traditional Optimisation must typically be re-solved when conditions change; Constrained RL adapts in real time using the already-trained policy.
- Dimension: Computation at Decision Time -- Traditional Optimisation can be computationally expensive to re-solve for large/dynamic problems; Constrained RL requires heavy computation only during training, decision time is fast (a forward pass).
- Dimension: Handling Uncertainty -- Traditional Optimisation generally assumes deterministic or simplified stochastic parameters; Constrained RL naturally learns from stochastic, noisy real-world feedback.
- Dimension: Optimality Guarantee -- Traditional Optimisation can provide provable optimal/near-optimal solutions for well-defined problems; Constrained RL provides no strict optimality guarantee, but a good approximate policy across many scenarios.
- Dimension: Interpretability -- Traditional Optimisation solutions/constraints are explicit and explainable; Constrained RL policies (especially deep-learning based) are comparatively less interpretable.

• Dimension: Scalability to Dynamic, Large-Scale Systems -- Traditional Optimisation scales poorly with frequent re-solving; Constrained RL scales well once trained, since inference is fast regardless of problem recurrence.

5. Assessment -- Which Method Provides Better Adaptability?

For a manufacturing plant operating under variable resource limitations and facing frequent real-world disruptions (machine breakdowns, urgent orders, fluctuating demand), constrained Reinforcement Learning provides better adaptability in dynamic production environments, because its learned policy can respond immediately to a changed state without being fully re-optimised, unlike traditional optimisation methods which require re-solving the model each time conditions shift. However, traditional optimisation remains preferable, or a better complement, for stable, well-characterised sub-problems (e.g., an initial daily schedule under known, fixed conditions) where a provably optimal solution is required and available computation time permits a full re-solve.

6. Decision / Recommendation

A hybrid approach is recommended in practice: use traditional optimisation (e.g., MIP) to generate the baseline daily/shift schedule under known conditions, and use a constrained RL agent as a real-time adjustment layer that reacts to unexpected disruptions (breakdowns, urgent orders) between full re-optimisation cycles, combining the optimality strength of classical methods with the adaptability strength of constrained RL.

7. Conclusion

Traditional optimisation and constrained Reinforcement Learning represent complementary rather than strictly competing paradigms: the former excels at provably optimal solutions for static, well-defined problems, while the latter excels at real-time adaptability in dynamic, uncertain production environments, making constrained RL the more suitable choice when a manufacturing plant must continuously adapt its schedule to changing operational conditions.

Code of Conduct:

*I **Charandeep N C (192472196)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Charandeep N C

RUBRICS FOR CALCULATION

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand